

Q1. [Asymptotic Notations]

- (a) (i) $O(n^{0.7})$
(ii) $O(2^{5n})$
(iii) $O(n(n!))$
(iv) $O(n^2)$
(v) $O(n \log_2 \log_2 n)$

- (b) (i) $\Theta(n^{0.7})$
(ii) $\Theta(2^{5n})$
(iii) $\Theta(n(n!))$
(iv) $\Theta(n^2)$
(v) $\Theta(n \log_2 \log_2 n)$

- (c) (i) $\Omega(n^{0.7})$
(ii) $\Omega(2^{5n})$
(iii) $\Omega(n(n!))$
(iv) $\Omega(n^2)$
(v) $\Omega(n \log_2 \log_2 n)$

- (d) Proof $\exists c=2000, n_0=1 \bullet \forall n > n_0 \bullet$

$$\begin{aligned} n &\geq 1 \\ n^3 &\leq n^4 \\ 2000n^3 &\leq 2000n^4 \\ 2000n^3 &\leq cn^4. \\ 2000n^{100} &\leq cn^{101} \end{aligned}$$

- (e) We prove $[\exists c, n_0 \bullet \forall n > n_0 \bullet 2000n^{100} \geq cn^{101}]$ is wrong.
It is equivalent to prove $\neg[\exists c, n_0 \bullet \forall n > n_0 \bullet 2000n^{100} \geq cn^{101}]$ is wrong.
It is equivalent to prove $\forall c, n_0 \bullet \exists n > n_0 \bullet 2000n^{100} < cn^{101}$.
Proof: $\forall c, n_0 \bullet \exists n = \max(n_0, \lceil 2000/c \rceil) + 1 > n_0 \bullet 2000n^{100} < cn^{101}$

$$\begin{aligned} n &> (2000/c) \\ cn &> 2000 \\ cn^{101} &> 2000n^{100} \\ 2000n^{100} &< cn^{101}. \end{aligned}$$

Q2. [Solving Recurrences]

- (a) (i)
 $a = 3, b=4, f(n)=n \log n$.
 $\exists \epsilon=1 - (\log_4 3) > 0 \bullet$
 $f(n) = n \log n > n = n^{(\log_4 3 + \epsilon)} = n^{(\log_b a + \epsilon)}$
Regularity checking
 $\exists c = 3/4 < 1 \bullet$
 $af(n/b) = 3f(n/4) = 3(n/4) \log(n/4) = (3/4)n \log(n/4) < (3/4)n \log n = cf(n)$
Therefore, case 3 applies.
 $T(n) = \Theta(n \log n)$.

- (ii)
 $a = 4, b=4, f(n)=n \log n$.

$\exists k=1 \bullet$

$$f(n) = n \log n = n \log^k n = n^{(\log_4 4)} \log^k n = n^{(\log_b a)} \log^k n$$

Case 2 applies.

$$T(n) = \Theta(n \log^{k+1} n) = \Theta(n \log^2 n)$$

(iii)

$$a = 5, b = 4, f(n) = n \log n.$$

$$\exists \epsilon = (\log_4 5 - 1)/2 > 0 \bullet$$

$$f(n) = n \log n < n^{(\log_4 5 - \epsilon)} = n^{(\log_b a - \epsilon)}$$

Therefore, case 1 applies.

$$T(n) = \Theta(n^{(\log_4 5)}).$$

(b) (i)

Claim: $T(n) < cn$, where $c > 1$, and $n > (7c / (c-1))^2$.

$$T(n) = 7T(\sqrt{n}) + n < 7c\sqrt{n} + n < cn,$$

because

$$n > (7c / (c-1))^2 \Rightarrow \sqrt{n} > 7c / (c-1) \Rightarrow 7c < (c-1)\sqrt{n} \Rightarrow 7c\sqrt{n} < (c-1)n \\ \Rightarrow 7c\sqrt{n} + n < cn$$

(ii)

Claim $T(n) < c n^3 \log n$, $c > 1$, $n > 2$

$$T(n) = 8T(n/2) + n^3 \\ < 8c(n/2)^3 \log(n/2) + n^3 \\ = cn^3 \log(n/2) + n^3 \\ = cn^3 \log n - cn^3 \log 2 + n^3 \\ < cn^3 \log n$$

(iii)

Claim $T(n) < c n^2 - c'n$, $c > c'$, $c' > 1/2$, $n > 3$

$$T(n) = 9T(n/3) + n \\ < 9[c(n/3)^2 - c'(n/3)] + n \\ = cn^3 - 3c'n + n \\ = cn^3 - c'n - 2c'n + n \\ = cn^3 - c'n - (2c'n - n) \\ < cn^3 - c'n$$

(c) Yes, by using another function.

No, the format does not match.

Q3. [Design and Analysis of Algorithm]

(a) The best case scenario is that the elements in L1 are all less than those in L2. Then, just connect the last element in L1 with the first element in L2 in constant time.

(b) Create a new linked list R, which is initially empty. $O(1)$ -time
While L1 and L2 are both non-empty $O(m+n)$ -time
 If the 1st element of L1 < the 1st element of L2
 Take away the 1st element from L1 and append it to R
 Else (ie. If the 1st element of L1 $\geq$ the 1st element of L2)
 Take away the 1st element from L2 and append it to R

If L1 is not empty $O(1)$ -time

 Append L1 to R

If L2 is not empty $O(1)$ -time

- (c) If the last element in L1 < the first element in L2
 Connect L2 to the end of L1.
 Use the head pointer of L1 as that of the new linked list.
 Else If the last element in L2 < the first element in L1
 Connect L1 to the end of L2.
 Use the head pointer of L2 as that of the new linked list.
 Else Perform the algorithm in (b).
- (d) The best case time complexity is a constant c , with probability 0.999.
 The worst case time complexity is $c'(m+n)$, with probability 0.001.
 Since there is no other cases,
 the expected time complexity is $0.999c + 0.001c'(m+n) = O(m+n)$.

Q4. [Binary Tree]

- a) A node has a pointer pointing to its first child, and its i th child is only pointed by its $(i-1)$ th child. The node also has a pointer pointing to its next sibling.
- b) Let n be the #node of a binary tree, and n_L and n_R are the #node's of the left and right subtrees, respectively.
 Let $T(n)$ be the time complexity of preorder traversal of an n -node binary tree.

$T(n) = T(n_L) + T(n_R) + 1$
 The term "+1" is for the overhead.

- c) $O(n)$.
 Use inorder traversal to output the integers will give sorted output.
 Binary search tree is already a sorted structure, and the inorder traversal is not a sorting algorithm.
- d) Buildtree(i, j) // construct a subtree from input $A[i..j]$
 If $i > j$ then return NULL;
 // else if $i \leq j$
 Create a node R , and set $R.x = A[i]$;
 If $i = j$ then
 $R.left = R.right = \text{NULL}$;
 else
 Use $A[i]$ to partition $A[i+1..j]$ into $A[i+1..k]$ and $A[k+1..j]$ such that
 $A[i]$ is greater than any elements in $A[i+1..k]$,
 and no greater than any elements in $A[k+1..j]$;
 $R.left$ points to Buildtree($i+1, k$);
 $R.right$ points to Buildtree($k+1, j$);
 Return R ;
-